

Ontwerp en realisatie van een performante provisioner voor een virtuele Windows-en Linux omgevingen in Rust.

Jens Gosseye.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. B. Van Vreckem

Co-promotor: Dhr. S. Beeckman

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Binnen de opleiding Toegepaste Informatica aan HOGENT wordt in de laatste 2 jaren gewerkt met geautomatiseerde omgevingen om virtuele Windows- en Linux-systemen klaar te zetten. Hierbij wordt gebruikgemaakt van Vagrant, een tool die het mogelijk maakt om labo- en ontwikkelomgevingen op een declaratieve manier aan te maken en provisionen. In de praktijk blijkt echter dat dit proces niet altijd even vlot verloopt. Opstarttijden kunnen traag zijn, provisioning kan vastlopen en bij onderbrekingen of foutmeldingen blijft het proces soms op de achtergrond doorlopen. Dit zorgt voor frustratie en tijdverlies bij studenten en docenten.

Daarom heb ik voor deze bachelorproef gekozen om een eigen provisioner te ontwikkelen in Rust. De keuze voor dit onderwerp sluit aan bij mijn interesse in automatisering en programmeren, ook al volgt deze bachelorproef niet rechtstreeks uit mijn gekozen afstudeerrichting. Een provisioner combineert toch programmeren met systeembeheer en infrastructuur, waardoor het onderwerp goed aansluit bij de bredere richting van toegepaste informatica. Bovendien leek het interessant om na te gaan of een eigen oplossing performanter en gebruiksvriendelijker kan zijn dan de bestaande aanpak met Vagrant.

Samenvatting

Deze bachelorproef behandelt het ontwerp en de realisatie van RustProv, een provisioner voor virtuele Windows- en Linuxomgevingen. Binnen de opleiding Toegepaste Informatica aan HOGENT wordt voor het opzetten van labo- en ontwikkelomgevingen vaak gebruikgemaakt van Vagrant, maar in de praktijk leidt dit soms tot trage opstarttijden, foutgevoelige provisioning en beperkte controle over het proces. Vanuit dit punt werd onderzocht hoe een eigen provisioner in Rust ontwikkeld kan worden die beter aansluit bij de behoeften van een onderwijsomgeving. Het doel van deze bachelorproef is een proof-of-concept te realiseren dat virtuele machines automatisch kan aanmaken, configureren en provisionen op een consistente en betrouwbare manier. Daarbij wordt ook gekeken naar praktische verbeteringen, zoals het direct koppelen van een VDI-bestand, het gebruik van shared folders en het ondersteunen van zowel één VM als meerdere VM's. De performantie van RustProv wordt bovendien vergeleken met die van Vagrant om na te gaan of de nieuwe aanpak sneller of efficiënter is. Deze bachelorproef levert daarmee een werkende basis op voor verdere uitbreiding van geautomatiseerde VM-provisioning in een onderwijscontext.

Inhoudsopgave

Lijst van figuren	vii
Lijst van tabellen	viii
Lijst van codefragmenten	ix
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	1
1.3 Onderzoeksdoelstelling	2
1.4 Opzet van deze bachelorproef	2
2 Stand van zaken	3
2.0.1 RustProv	3
2.0.2 Virtualisatie en provisioning	3
2.0.3 Waarom een provisioner?	3
2.0.4 Infrastructure as Code	3
2.0.5 Educatieve context	4
2.0.6 Rust als programmeertaal	4
2.0.7 Afbakening van de doelomgeving	4
2.0.8 Tijdswinsten tijdens het provisioningsproces	5
2.0.9 Provisioningworkflow	5
2.0.10 Provisioningworkflow	5
2.0.11 Functionele uitbreidingen	7
2.0.12 Ondersteuning	7
2.0.13 Dependencies	8
2.0.14 Snelheidsvergelijking met Vagrant	9
3 Methodologie	11
3.0.1 Fase 1: Rust aanleren	11
3.0.2 Fase 2: Sprint 1 - opstarten	11
3.0.3 Fase 3: Sprint 2 - provisionen	12
3.0.4 Fase 4: Sprint 3 - configuratie	12
3.0.5 Fase 5: Sprint 4 - functies	12
3.0.6 Fase 6: Sprint 5 - foutafhandeling	13
3.0.7 Planning	13

4 Conclusie	14
A Onderzoeksvoorstel	16
A.1 Inleiding	16
A.2 Literatuurstudie	17
A.2.1 Infrastructure as Code	17
A.2.2 Educatieve context	17
A.2.3 Rust als programmeertaal	18
A.3 Methodologie	18
A.3.1 Fase 1: Rust aanleren	18
A.3.2 Fase 2: Sprint 1 - opstarten	18
A.3.3 Fase 3: Sprint 2 - provisionen	18
A.3.4 Fase 4: Sprint 3 - configuratie	19
A.3.5 Fase 5: Sprint 4 - functies	19
A.3.6 Fase 6: Sprint 5 - foutafhandeling	19
A.3.7 Planning	19
A.4 Verwacht resultaat, conclusie	19
Bibliografie	20

Lijst van figuren

A.1 Gantt diagram met de verschillende fasen en milestones van het onderzoek.	18
---	----

Lijst van tabellen

2.1	Vergelijking van de Windows-tijden tussen Vagrant en RustProv in seconden.	10
2.2	Vergelijking van de Linux-tijden tussen Vagrant en RustProv in seconden.	10

Lijst van codefragmenten

2.1 Zoeken naar de gebruikersmap per besturingssysteem	8
--	---

1

Inleiding

Binnen de richting IT Infrastructure Engineer aan HOGENT wordt gebruikgemaakt van verschillende provisioningtools om virtuele Windows- en Linuxomgevingen op te zetten. Een veelgebruikte tool is Vagrant: studenten gebruiken deze om ontwikkel- en labo-omgevingen te automatiseren op basis van een declaratieve configuratiefile. In de praktijk treden echter verschillende problemen op tijdens dit proces: virtuele machines starten traag op, het provisioningproces loopt soms vast of wordt niet gestart door time-outs, virtuele machines kunnen onverwachte *kernel panics* vertonen en bij het onderbreken van het provisioningproces blijft dit soms op de achtergrond verder draaien, waardoor de gebruiker het proces manueel moet stopzetten. Dit zorgt voor veel frustraties en tijdverlies, zowel in onderwijs- als bedrijfscontext.

1.1. Probleemstelling

De huidige Vagrant-gebaseerde aanpak voor provisioning in de opleiding Toegepaste Informatica aan HOGENT leidt regelmatig tot trage opstarttijden, vastlopende provisioning en instabiele virtuele machines. Dit zorgt ervoor dat studenten en lectoren tijd verliezen tijdens de lessen, labo's en evaluaties. Hierdoor wordt de focus van de lessen verlegd van het leren naar het oplossen van verschillende technische problemen. De doelgroep van dit onderzoek bestaat uit studenten en docenten binnen de richting IT Infrastructure Engineer aan HOGENT. Zij hebben nood aan een meer performante en betrouwbare provisioningoplossing die beter is afgestemd op een onderwijs- en laboomgeving.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag van deze bachelorproef luidt:

Hoe ontwerp en realiseer je een performante provisioner voor virtuele Windows- en Linuxomgevingen, die beter inspeelt op de behoeften van onderwijs en bedrijven dan de huidige Vagrant-gebaseerde aanpak?

Om deze hoofdonderzoeksvraag te concretiseren, worden volgende deelvragen onderzocht:

- Welke functionaliteiten zijn essentieel voor een provisioner die in onderwijs- en labo-omgevingen wordt gebruikt?
- Op welke besturingssystemen zal de provisioner zelf draaien?
- Voor welke gast-besturingssystemen moet de provisioner virtuele machines kunnen opbouwen en configureren?

1.3. Onderzoeksdoelstelling

Het doel van deze bachelorproef is een proof-of-concept provisioner te schrijven in Rust. Deze zal virtuele Linux- en Windows-omgevingen kunnen opzetten op een betrouwbare manier. De succescriteria omvatten een werkende command-line tool die op Windows kan worden uitgevoerd, en ondersteuning voor kernfunctionaliteiten zoals het opstarten van de VM, provisioning via SSH en configuratie via een declaratief bestand. Daarnaast wordt een aantoonbare verbetering in de provisioningtijd en stabiliteit tegenover de bestaande Vagrant-omgevingen verwacht. Tot slot zal er documentatie en een gebruikershandleiding voorzien worden, zodat de tool inzetbaar is binnen de opleiding Toegepaste Informatica aan HOGENT.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 4, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

2.0.1. RustProv

RustProv is de naam van de provisioner die in deze bachelorproef wordt ontwikkeld. De naam combineert *Rust*, de gekozen programmeertaal, en *Prov*, een afkorting van provisioning.

2.0.2. Virtualisatie en provisioning

Virtuele machines vormen de basis van de omgevingen waarvoor de provisioner wordt ontwikkeld. In deze bachelorproef ligt de focus op het automatisch aanmaken, configureren en beschikbaar maken van een VM in een onderwijscontext. Virtualisatie laat toe om meerdere geïsoleerde omgevingen op dezelfde fysieke machine te gebruiken, wat handig is voor labo's, testen en demo-omgevingen.

2.0.3. Waarom een provisioner?

Een provisioner automatiseert het opzetten en configureren van virtuele omgevingen. Daardoor verlopen testen en deployments consistent, sneller en met minder kans op fouten. In deze bachelorproef is een provisioner vooral nuttig om een stabiele en herhaalbare omgeving te voorzien voor onderwijs- en labo-opdrachten. Dit vermijdt ook dat dezelfde configuratie telkens manueel opnieuw moet worden uitgevoerd.

2.0.4. Infrastructure as Code

Het automatiseringsproces in systeembeheer wordt vaak aangeduid als *Infrastructure as Code (IaC)*. Binnen IaC wordt een onderscheid gemaakt tussen verschillende soorten tools, zoals *provisioning tools*, *configuration management* en *orchestration*. Provisioningtools richten zich op het creëren van de infrastructuur, terwijl configuratiemanagementtools instaan voor de installatie en configuratie van

software op de server. Het voordeel hiervan is dat de opbouw van een omgeving reproduceerbaar en beter beheersbaar wordt.

Voor lokale ontwikkelomgevingen worden vaak tools zoals Vagrant gebruikt. Daarnaast bestaan er tools zoals Ansible en Terraform, die eerder geassocieerd worden met cloudomgevingen of grotere configuraties. Welke tool gekozen wordt, hangt af van de context, de grootte van de omgeving en de mate van automatisering die nodig is.

2.0.5. Educatieve context

Hoewel er verschillende commerciële oplossingen bestaan voor enterprise-omgevingen, gelden in een onderwijscontext andere eisen voor virtualisatie. Onderzoek naar het gebruik van virtuele machines in het onderwijs toont dat een evenwicht nodig is tussen prestatie, gebruiksvriendelijkheid voor studenten en de beperkingen van consumentenhardware zoals laptops. In een labo-omgeving moet een virtuele machine dus niet alleen werken, maar ook snel beschikbaar zijn.

Ook speelt tijd een grote rol. De *provisioning time*, de tijd tussen het aanvragen van de virtuele machine en de beschikbaarheid ervan, is een kritieke performance-indicator die de productiviteit beïnvloedt. Lange provisioning times kunnen tijdens labo's of examens leiden tot minder effectieve werktijd. Daardoor blijft de vraag bestaan hoe een lichtgewicht en snelle provisioner gerealiseerd kan worden die specifiek afgestemd is op Windows- en Linuxomgevingen in een educatieve context. Dit is extra relevant wanneer meerdere studenten tegelijk met dezelfde tool moeten werken.

2.0.6. Rust als programmeertaal

Rust werd gekozen als programmeertaal voor de provisioner vanwege de goede cross-platform compatibiliteit, waardoor de tool zowel op Linux als op Windows kan draaien zonder grote aanpassingen. De taal combineert prestatie met geheugenveiligheid via het ownership-model, waardoor veelvoorkomende fouten zoals geheugenlekken worden vermeden. Dit is belangrijk voor infrastructuurtools, omdat fouten zoals een onderbroken provisioning of een mislukte SSH-verbinding gecontroleerd moeten kunnen worden afgehandeld. Rust is relatief jong, maar wordt steeds vaker gebruikt voor systeemsoftware en tools waar betrouwbaarheid belangrijk is.

2.0.7. Afbakening van de doelomgeving

De provisioner wordt ontworpen voor de concrete SEP-opdracht en moet de stappen ondersteunen die nodig zijn om die opdracht in een vooraf opgezette virtuele omgeving uit te voeren. Dit maakt de tool doelgericht en beperkt de scope tot wat effectief nodig is voor de opleiding. Door die afbakening blijft het project haalbaar binnen de beschikbare tijd en blijft de focus liggen op een werkende proof-of-

concept. Dit project gebruikt een bridge adapter voor het netwerk en drie test-VM's die eerder voor deze opdracht werden ingezet.

2.0.8. Tijdswinsten tijdens het provisioningsproces

VDI-gebaseerde opstart

In plaats van een standaard VirtualBox-image wordt de virtuele schijf rechtstreeks gekoppeld via een VDI-bestand. Dit kan de opstart en provisioning vereenvoudigen doordat er minder extra conversie- of opbouwstappen nodig zijn. Het importeren van een volledige VM kan lang duren, terwijl het koppelen van een bestaande VDI dit proces mogelijk versnelt. Daardoor wordt de omgeving sneller beschikbaar voor gebruik.

Een belangrijk ontwerpdoel is het verkorten van de provisioningtijd. Door een VDI rechtstreeks te koppelen in plaats van met een volledig opgebouwde VirtualBox-image te werken, kan de opstartprocedure mogelijk sneller verlopen. Dit moet later bevestigd worden door een vergelijking tussen verschillende configuraties. De snelheid van dit proces is belangrijk omdat de gebruiker zo minder lang hoeft te wachten vooraleer de omgeving klaar is.

VM's eerst opstarten

Bij het gebruik van meerdere virtuele machines worden eerst alle VM's opgestart voordat de scripts worden uitgevoerd. Hierdoor kan de totale wachttijd verminderd worden, omdat de opstarttijd van de virtuele machine en de SSH-beschikbaarheid niet per machine apart moet worden afgewacht tijdens de provisioning.

Netwerk- en opslagopzet

Naast de VDI-keuze speelt ook de netwerkconfiguratie een rol in de totale opstarttijd. Door de VM meteen via een bridge adapter of vergelijkbare netwerkopzet te verbinden, kan het systeem sneller bereikbaar worden voor verdere provisioning. Dit vermindert de tijd tussen het opstarten van de VM en het moment waarop de provisioner effectief kan beginnen.

2.0.9. Provisioningworkflow

De provisioner doorloopt een vaste reeks stappen: de VM wordt opgestart, het IP-adres wordt opgehaald, er wordt verbinding gemaakt via SSH en vervolgens worden de provisioningstaken uitgevoerd. Deze reeks gebeurt in een vaste volgorde omdat elke stap afhankelijk is van de vorige. Zonder een correct opgestarte VM kan bijvoorbeeld geen netwerkverbinding worden gemaakt, en zonder SSH kan geen verdere configuratie gebeuren.

2.0.10. Provisioningworkflow

De provisioner doorloopt een vaste reeks stappen: de VM wordt opgestart, het IP-adres wordt opgehaald, er wordt verbinding gemaakt via SSH en vervolgens wor-

den de provisioningstaken uitgevoerd. Deze reeks gebeurt in een vaste volgorde omdat elke stap afhankelijk is van de vorige. Zonder een correct opgestarte VM kan bijvoorbeeld geen netwerkverbinding worden gemaakt, en zonder SSH kan geen verdere configuratie gebeuren.

1. **Aanmaken VM**

In deze stap wordt de virtuele machine voorbereid en geregistreerd in de virtualisatieomgeving. Dit vormt de basis van het volledige provisioningproces. Daarna kunnen de netwerk- en opslaginstellingen worden aangepast zodat de machine correct kan starten.

2. **Kopiëren van de VDI en aanpassing van de SID**

Hier wordt de virtuele schijf gekopieerd of hergebruikt en worden de nodige instellingen aangepast zodat de VM correct kan opstarten. Deze stap zorgt ervoor dat de nieuwe omgeving een eigen identiteit krijgt en niet conflicteert met andere virtuele machines. Ook kunnen hier eventueel machine-specifieke instellingen worden ingesteld.

3. **Bridgeadapter**

Er wordt gebruikgemaakt van een bridgeadapter, zodat een tweede netwerk-interface aan de VM wordt toegevoegd. Deze keuze werd gemaakt omwille van de eenvoud. Het toevoegen van extra netwerkadapters zou de opzet complexer maken en extra elementen introduceren die voor deze benchmarks niet nodig zijn.

4. **Shared folders**

In deze fase wordt de `./script`-map gemount aan de virtuele machine. Hierdoor worden bestanden en scripts van de host direct toegankelijk binnen de VM. De shared folder wordt gebruikt zodat de provisioner, eenmaal verbonden via SSH, de benodigde scripts kan uitvoeren die zich in deze map bevinden.

5. **NAT Port Forwarding**

De netwerkconfiguratie wordt aangepast zodat de host toegang krijgt tot de virtuele machine via port forwarding. Dit is nodig om later via SSH verbinding te kunnen maken. Zonder deze stap zou de provisioner geen betrouwbare communicatie met de VM kunnen opzetten.

6. **Opstarten van de VM**

Na de configuratie wordt de virtuele machine effectief opgestart. Op dit moment wordt gecontroleerd of de VM correct reageert en of de benodigde netwerkparameters beschikbaar zijn. Deze stap is essentieel voordat de provisioning verder kan gaan.

7. SSH en shared folder

Zodra de VM beschikbaar is, wordt via SSH verbinding gemaakt en kunnen gedeelde mappen of andere provisioningstaken worden uitgevoerd. Via SSH kunnen configuratiescripts of installatieopdrachten op afstand worden uitgevoerd. Shared folders kunnen daarnaast gebruikt worden om bestanden tussen host en VM uit te wisselen.

2.0.11. Functionele uitbreidingen

Naast het opstarten en provisionen voorziet de tool ook extra functies die het gebruik in labo-omgevingen verbeteren, zoals snapshots, foutafhandeling en logging. Deze functies maken het systeem robuuster en praktischer voor dagelijks gebruik. Ze zorgen er ook voor dat de gebruiker sneller kan herstellen wanneer er iets misloopt.

Snapshotting

Voor het provisionen kan een snapshot worden gemaakt, zodat de gebruiker bij fouten altijd kan terugkeren naar een stabiele toestand. Dit verhoogt de betrouwbaarheid, maar kost wel extra opslagruimte. In een onderwijscontext kan dit nuttig zijn om snel opnieuw te beginnen zonder alles opnieuw te moeten installeren.

Priority Queue

De VM's kunnen in een vaste volgorde worden opgestart via een prioriteitsmechanisme. Die volgorde wordt als extra parameter in het configuratiebestand opgenomen. Dit kan handig zijn wanneer bepaalde machines afhankelijk zijn van andere machines die eerst beschikbaar moeten zijn.

2.0.12. Ondersteuning

Ondersteunde besturingssystemen

De code past zich automatisch aan op basis van het besturingssysteem waarop de provisioner wordt uitgevoerd. Hierdoor kan de juiste gebruikersmap of tijdelijke opslaglocatie worden gebruikt, afhankelijk van of de tool op Windows of Linux draait.

```
1 pub fn get_home_dir() -> std::path::PathBuf {  
2     std::env::var("HOME")  
3     .map(std::path::PathBuf::from)  
4     .or_else(|_| {  
5         std::env::var("USERPROFILE")  
6         .map(std::path::PathBuf::from)  
7         .map_err(|_|  
8             std::io::Error::new(std::io::ErrorKind::NotFound, "Home  
9             directory not found"))  
10    })  
11    .expect("Failed to determine home directory")  
12 }
```

Codefragment 2.1: Voorbeeld van hoe de doelmap dynamisch wordt opgebouwd op basis van het besturingssysteem.

Ondersteunde besturingssystemen voor provisioning

Er werd gekozen voor ondersteuning van Linux en Windows, omdat dit de twee besturingssysteemtypes zijn die binnen HOGENT het vaakst gebruikt worden. Voor deze bachelorproef werden vier boxes voorzien.

AlmaLinux

AlmaLinux is de gekozen Linuxdistributie binnen HOGENT voor het vak Linux. Daarom wordt deze distributie ook ondersteund binnen de provisioner.

Ubuntu

Ubuntu is een van de meest gebruikte Linuxdistributies en werd daarom opgenomen als ondersteunde omgeving. [//https://www.openlogic.com/blog/top-enterprise-linux-distributions](https://www.openlogic.com/blog/top-enterprise-linux-distributions)

Windows 10

Windows 10 werd gekozen als clientbesturingssysteem voor de Windowsvakken.

Windows Server 2025

Windows Server 2025 werd gekozen als serverbesturingssysteem voor de Windowsvakken.

één voor AlmaLinux, één voor Ubuntu en twee voor Windows-omgevingen. De Windows boxes zijn voor Windows 10 en voor Windows Server 2025

2.0.13. Dependencies

Voor de implementatie worden onder andere de volgende Rust-crates gebruikt:

- `ssh2` voor SSH-verbindingen. Dit werd gekozen voor cross-platform compatibiliteit, omdat de ingebouwde SSH-oplossingen van Linux en Windows te verschillend zijn en niet op dezelfde manier het wachtwoord van de gebruiker kunnen verwerken. De crate is een Rust-binding voor `libssh2`, een SSH-clientbibliotheek, en is dual-licensed onder Apache 2.0 en MIT. (**ssh2Crate**)
- `toml` voor het inlezen van configuratiebestanden. Deze crate werd gekozen omwille van de eenvoud en duidelijkheid van het importeren van data uit een TOML-bestand. TOML is een leesbaar configuratieformaat dat vlot kan worden omgezet naar Rust-structuren. De crate is een serde-compatibele TOML-decoder en encoder voor Rust en is dual-licensed onder Apache 2.0 en MIT. (**tomlCrate**)
- `clap` voor command-line argumenten. Dit werd gekozen omdat het een eenvoudige en gebruiksvriendelijke manier biedt om CLI-argumenten te verwerken. De crate is dual-licensed onder Apache 2.0 en MIT. (**clapCrate**)

2.0.14. Snelheidsvergelijking met Vagrant

De uiteindelijke implementatie wordt vergeleken met Vagrant om na te gaan of de nieuwe provisioner sneller of trager is. Hiervoor wordt de provisioningtijd gemeten onder gelijkaardige omstandigheden. De vergelijking gebeurt op basis van een beperkt aantal testgevallen, namelijk drie Linux-VM's en twee Windows-VM's. Op die manier kan worden nagegaan of de gekozen aanpak een echte verbetering vormt ten opzichte van de bestaande oplossing.

Deze testen werden uitgevoerd op een computer met 32 GB DDR4-geheugen aan 3200 MHz, een AMD Ryzen 7 5800H-processor en een NVIDIA GeForce RTX 3060-laptopgpu met een vermogen van 90 W op Windows 11 25H2.

- **Eén VM:** Tijd vanaf het opstarten van de VM tot de start van het provisioning-proces. Deze meting wordt stopgezet op het moment dat de provisioning begint, omdat dit een beter beeld geeft van de snelheid van de provisioner zelf. Het uitvoeren van het provisioningscript wordt namelijk mede beïnvloed door externe factoren zoals internetverbindingen, wat de meting minder stabiel maakt.
- **Meerdere VM's:** Totale uitvoertijd van alle VM's, na elkaar uitgevoerd. Deze meting is realistischer omdat ook de uitvoering van de scripts wordt meegerekend, waardoor een meer volledige vergelijking met meerdere VM's mogelijk is.

Windows

Voor Windows worden een Windows Server 2025-omgeving en een Windows 10-clientomgeving getest. De Windows-gevallen zijn gebaseerd op de SEP-opdracht

van vorig jaar en bevatten een domeincontroller en een client die via DHCP wordt toegevoegd aan het domein. Eerst wordt de tijd gemeten tot de provisioning van één VM volledig afgerond is. Daarna wordt ook de totale tijd gemeten wanneer meerdere VM's worden opgestart en alle scripts volledig zijn uitgevoerd.

Test	Vagrant	RustProv
Eén VM: provisioningtijd	x	x
Meerdere VM's: totale uitvoeringstijd	x	x

Tabel 2.1: Vergelijking van de Windows-tijden tussen Vagrant en RustProv in seconden.

Linux

Voor Linux worden drie virtuele machines op een gelijkaardige manier geanalyseerd. Ook hier wordt gekeken naar de tijd tot de provisioning van één VM volledig afgerond is. Vervolgens wordt de totale uitvoeringstijd gemeten voor het scenario met meerdere VM's, waarbij alle scripts uitgevoerd en afgerond zijn. Op die manier kan een eerlijke vergelijking gemaakt worden tussen Vagrant en RustProv.

Test	Vagrant	RustProv
Eén VM: provisioningtijd	x	x
Meerdere VM's: totale uitvoeringstijd	x	x

Tabel 2.2: Vergelijking van de Linux-tijden tussen Vagrant en RustProv in seconden.

3

Methodologie

3.0.1. Fase 1: Rust aanleren

In de eerste fase wordt de focus gelegd op het aanleren van de gekozen programmeertaal Rust. Hierbij zullen de basisconcepten van de taal geleerd worden, zoals functies, methodes, datatypes en algemene syntax. Aan de hand van verschillende kleine projecten worden deze concepten dan toegepast en aangeleerd.

Week 1-2

Tijdens deze week werd de Rust Quick Start Guide (**Arbuckle2018**) doorgenomen en werden er verschillende scripts uitgevoerd om diverse concepten uit te testen.

3.0.2. Fase 2: Sprint 1 - opstarten

In de eerste sprint ligt de focus op het kunnen opstarten van de virtuele machine (VM) op basis van een gemaakte testbox. De provisioner zal een VM kunnen aanmaken, configureren en starten met het virtualisatieplatform VirtualBox.

Week 3

Tijdens deze week werd het opstarten van de VM aan de hand van verschillende VBoxManage-commando's geïmplementeerd. Dit begon met het maken van een Ubuntu- en Windows 10-VM om als basisboxen te gebruiken. Via het script wordt een nieuwe VM aangemaakt met het gekozen besturingssysteem. Dit wordt gedaan door het .vdi-bestand te kopiëren van de vooraf aangemaakte VM en de Universally Unique Identifier (UUID) aan te passen, zodat er geen conflicten optreden. Vervolgens wordt deze schijf gemount en wordt de VM opgestart, nu deze klaar is voor verder gebruik.

Week 4

Tijdens deze week werd de code herschreven zodat die meer overeenkomt met Objectgeoriënteerd programmeren (OOP). Dit maakt de structuur duidelijker en

vergemakkelijk toekomstige uitbreidingen van de code.

3.0.3. Fase 3: Sprint 2 - provisioning

In de tweede stap wordt het provisioningproces verder uitgewerkt. Hierbij wordt de NAT-portforwarding aangepast zodat er via SSH verbinding kan worden gemaakt met de VM. Daarna worden de provisioningstappen uitgevoerd op het systeem.

Week 5

Tijdens deze week werd de virtuele machine verder aangepast zodat er aparte functies beschikbaar zijn voor Linux- en Windows-VM's. Hierdoor kan de uitvoering van scripts afhankelijk worden gemaakt van het type besturingssysteem.

Week 6

In deze week werd de code verder toegepast voor het verbinden met de VM via SSH en het uitvoeren van scripts op basis van het gebruikte besturingssysteem. Op die manier wordt het provisioningproces beter afgestemd op de gekozen omgeving.

3.0.4. Fase 4: Sprint 3 - configuratie

In de derde sprint wordt er ondersteuning toegevoegd voor een configuratiebestand in het TOML-formaat. De provisioner zal het configuratiebestand inlezen en verwerken, de gevraagde instellingen zullen het provisioninggedrag besturen.

Week 7

Tijdens deze week werd het lezen van het TOML-bestand opgestart. Op dat moment was de implementatie nog niet volledig uitgewerkt, maar wel al voldoende om de eerste structuur op te zetten.

Week 8

Tijdens deze week werd het inlezen van het TOML-bestand verder uitgewerkt en geïntegreerd in de werking van het programma. Daarnaast werd een init-functie toegevoegd die de juiste folderstructuur aanmaakt.

3.0.5. Fase 5: Sprint 4 - functies

In de vierde sprint zullen nieuwe functies worden toegevoegd, zoals in het aanmaken en terugzetten van snapshots. Dit zorgt er voor dat de gebruiker snel terug zal kunnen gaan naar een stabiele toestand, zoals na een mislukte provisioningrun. Ook zal tijdens deze sprint meerdere boxes aangemaakt worden.

Week 9

Tijdens deze week werd begonnen met het toepassen van snapshots. De code werd aangepast zodat het aanmaken van snapshots correct kon worden geïntegreerd in het provisioningproces. Daarnaast werden functies opgesplitst zodat ze

afzonderlijk kunnen worden gebruikt voor één VM of voor meerdere VM's. Tot slot werd de CLI-input verder verbeterd met behulp van `c\lap`(**clapCrate**).

Week 10

Tijdens deze week werd een aparte functie toegevoegd om de status van de VM op te halen, zodat gecontroleerd kan worden of deze actief is. Daarnaast werd de bridge network adapter toegevoegd, zodat deze kan worden gebruikt wanneer de gebruiker dat wenst. Tot slot werden de snapshotfuncties verder uitgewerkt en toegepast.

3.0.6. Fase 6: Sprint 5 - foutafhandeling

In de laatste sprint ligt de focus op het bouwen van foutafhandeling en rapportering. Fouten worden gelogd met duidelijke foutboodschappen en genoteerd in een log file. Als er een fout is moet er ook op een veilige manier een fallback-actie uitgevoerd worden, dit zorgt ervoor dat de gebruiker niet met een onbruikbare omgeving blijft zitten.

Week 11

x

Week 12

x

3.0.7. Planning

De totale duur van deze methodologie is gepland over een periode van 12 schoolweken tijdens semester 2 van het academiejaar 2025-2026.

- Week 1-2: Rust aanleren
- Week 3-4: Sprint 1
- Week 5-6: Sprint 2
- Week 7-8: Sprint 3
- Week 9-10: Sprint 4
- Week 11-12: Sprint 5

4

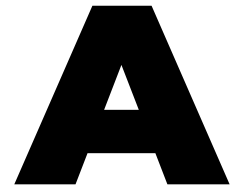
Conclusie

Curabitur nunc magna, posuere eget, venenatis eu, vehicula ac, velit. Aenean ornare, massa a accumsan pulvinar, quam lorem laoreet purus, eu sodales magna risus molestie lorem. Nunc erat velit, hendrerit quis, malesuada ut, aliquam vitae, wisi. Sed posuere. Suspendisse ipsum arcu, scelerisque nec, aliquam eu, molestie tincidunt, justo. Phasellus iaculis. Sed posuere lorem non ipsum. Pellentesque dapibus. Suspendisse quam libero, laoreet a, tincidunt eget, consequat at, est. Nullam ut lectus non enim consequat facilisis. Mauris leo. Quisque pede ligula, auctor vel, pellentesque vel, posuere id, turpis. Cras ipsum sem, cursus et, facilisis ut, tempus euismod, quam. Suspendisse tristique dolor eu orci. Mauris mattis. Aenean semper. Vivamus tortor magna, facilisis id, varius mattis, hendrerit in, justo. Integer purus.

Vivamus adipiscing. Curabitur imperdiet tempus turpis. Vivamus sapien dolor, congue venenatis, euismod eget, porta rhoncus, magna. Proin condimentum pretium enim. Fusce fringilla, libero et venenatis facilisis, eros enim cursus arcu, vitae facilisis odio augue vitae orci. Aliquam varius nibh ut odio. Sed condimentum condimentum nunc. Pellentesque eget massa. Pellentesque quis mauris. Donec ut ligula ac pede pulvinar lobortis. Pellentesque euismod. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent elit. Ut laoreet ornare est. Phasellus gravida vulputate nulla. Donec sit amet arcu ut sem tempor malesuada. Praesent hendrerit augue in urna. Proin enim ante, ornare vel, consequat ut, blandit in, justo. Donec felis elit, dignissim sed, sagittis ut, ullamcorper a, nulla. Aenean pharetra vulputate odio.

Quisque enim. Proin velit neque, tristique eu, eleifend eget, vestibulum nec, lacus. Vivamus odio. Duis odio urna, vehicula in, elementum aliquam, aliquet laoreet, tellus. Sed velit. Sed vel mi ac elit aliquet interdum. Etiam sapien neque, convallis et, aliquet vel, auctor non, arcu. Aliquam suscipit aliquam lectus. Proin tincidunt magna sed wisi. Integer blandit lacus ut lorem. Sed luctus justo sed enim.

Morbi malesuada hendrerit dui. Nunc mauris leo, dapibus sit amet, vestibulum et, commodo id, est. Pellentesque purus. Pellentesque tristique, nunc ac pulvinar adipiscing, justo eros consequat lectus, sit amet posuere lectus neque vel augue. Cras consectetur libero ac eros. Ut eget massa. Fusce sit amet enim eleifend sem dictum auctor. In eget risus luctus wisi convallis pulvinar. Vivamus sapien risus, tempor in, viverra in, aliquet pellentesque, eros. Aliquam euismod libero a sem. Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Binnen de richting IT infrastructure engineer wordt in HOGENT gebruikgemaakt van verschillende provisioningtools om virtuele Windows- en Linuxomgevingen op te zetten. Een veelgebruikte tool hiervoor is Vagrant, studenten gebruiken dit om een ontwikkel- en labo-omgeving te kunnen automatiseren op basis van een declaratieve configuratiefile.

In de praktijk komen er verschillende problemen tevoorschijn: de virtuele machines starten soms traag op, het provisioningproces hangt vast of kan niet gestart worden door time-outs, virtuele machines kunnen onverwachte kernel panics vertonen en als het provisioningproces onderbroken wordt tijdens uitvoer blijft het soms verder draaien op de achtergrond waardoor de gebruiker dit proces manueel moet stoppen om verder te kunnen werken. Dit zorgt voor vele frustraties en tijdverlies zowel binnen een onderwijscontext of bedrijfscontext.

De centrale onderzoeksvraag van deze bachelorproef is: “Hoe ontwerp en realiseer je een performante provisioner voor een virtuele Windows- en Linux omgeving, die beter inspeelt op de behoeften van onderwijs en bedrijven dan de huidige Vagrant-gebaseerde aanpak?” Om deze vraag te beantwoorden doen we aan de hand van deze deelvragen:

- Welke functionaliteiten zijn nodig voor een provisioner?
- Op welke besturingssystemen zal de provisioner gebruikt worden?
- Voor welke gast-besturingssystemen provisioned de tool?

Dit onderzoek streeft ervoor om een provisioner te maken die gebruikt kan worden door IT-professionals en studenten binnen een onderwijs- en labo-omgevingen. Om deze vragen te beantwoorden wordt een sprint-aanpak gebruikt, dit loopt over 12 weken: eerst studeren van de programmeertaal Rust, daarna stapsgewijs het kunnen opstarten van de Virtuele Machine, Provisioning via SSH, Configuratie via TOML, snapshots en foutafhandeling implementeren. Elke sprint bouwt verder op de vorige en zal getest worden aan de hand van testscenario's die gebaseerd zijn op HOGENT labo oefeningen

A.2. Literatuurstudie

Virtuele machines vormen de basis van veel Informaticatoepassingen. De voorname reden hiervoor is omdat ze het mogelijk maken om meerdere virtuele omgevingen te laten draaien op dezelfde fysieke hardware en zo de flexibiliteit, schaalbaarheid en resource-efficiëntie verhogen (Ahmed, 2020). In verschillende surveys wordt *provisioning* beschreven als het geautomatiseerd aanmaken, configureren en beheren van een virtuele machine (Abdulhamid et al., 2014; S P & Antony D'Mello, 2018). Zoals deze studies aangeven is het opzetten van een testomgeving met efficiënte provisioning-strategieën cruciaal bij het uitwerken van een project.

A.2.1. Infrastructure as Code

Het automatiseringsproces wordt in systeembeheer genoemd *Infrastructure as Code (IaC)*. Verschillende studies en rapporten binnen het IaC een maken onderscheid tussen verschillende soorten tools, zoals *emphprovisioning tools*, *configuration management* en *orchestration* (Brikman, 2025; Hill, 2015). Provisioningtools richten zich op het creëren van de infrastructuur, configuratiemanagement tools richten zich op de installatie en configuratie van software op de server (Ijaz, 2024). Het landschap van deze verschillende tools is divers. Voor de lokale ontwikkelingsomgevingen wordt gebruikt gemaakt van tools zoals Vagrant. Verschillende tools bestaan ook die dat niet gemaakt zijn als eerste instantie voor lokale omgevingen zoals Ansible en Terraform, deze worden vaak geassocieerd met cloud omgevingen of grote configuraties (Ismailzai, 2017).

A.2.2. Educatieve context

Alhoewel dat er veel verschillende commerciële oplossingen bestaan voor enterprise-omgevingen (Portworx, 2025), is er binnen een onderwijscontext andere verschillende eisen voor virtualisatie. Onderzoek naar het gebruik van virtuele machines (VM's) in het onderwijs toont dat een balans tussen performantie, gebruiksvriendelijkheid voor studenten en de beperkingen van consumenten hardware zoals laptops een complexe uitdaging is (Robison & Hacker, 2012). Ook speelt tijd een grote rol. De *provisioning time*, de tijd die tussen het aanvra-

Figuur A.1: Gantt diagram met de verschillende fasen en milestones van het onderzoek.

gen van de vm en beschikbaarheid ervan is kritieke performance-indicator (KPI) die dat productiviteit beïnvloed (KPI Depot, 2024). Lange provisioning time kan tijdens het maken van een labo of examen voor minder werktijd zorgen. Gelukkig zijn er wel verschillende algoritmen die dat dit kunnen versnellen (Lo et al., 2014; Luo et al., 2020), blijft de vraag bestaan hoe je een lichtgewicht en snelle provisioner kan gerealiseerd worden die dat specifiek gemaakt is voor Windows- en Linux-omgevingen voor een educatieve omgeving als alternatief voor de huidige Vagrant provisioner.

A.2.3. Rust als programmeertaal

Rust werd gekozen voor de provisioner vanwege goede cross-platform compatibiliteit, waardoor de tool zowel op Linux als Windows werkt zonder veel aanpassingen te doen. De taal combineert performantie en geheugenveiligheid door het Ownership-model, dit zorgt ervoor dat veel voorkomende fouten zoals geheugenlekken voorkomen worden (Steve Klabnik, 2025; Wylde, 2023). Dit is cruciaal voor infrastructuurtools, bij het afhandelen van foutscenario's zoals een onderbroken provisioning of een mislukte SSH-verbinding zorgt dit ervoor dat alles onder controle blijft (Wylde, 2023).

A.3. Methodologie

A.3.1. Fase 1: Rust aanleren

In de eerste fase wordt de focus gelegd op het aanleren van de gekozen programmeertaal Rust. Hierbij zullen de basisconcepten van de taal geleerd worden, zoals functies, methodes, datatypes en algemene syntax. Aan de hand van verschillende kleine projecten worden deze concepten dan toegepast en aangeleerd.

A.3.2. Fase 2: Sprint 1 - opstarten

In de eerste sprint ligt de focus op het kunnen opstarten van de virtuele machine (VM) op basis van een gemaakte testbox. De provisioner zal een VM kunnen aanmaken, configureren en starten met het virtualisatieplatform VirtualBox.

A.3.3. Fase 3: Sprint 2 - provisionen

In de tweede stap zal het provisioningproces worden uitgewerkt. De tool zal het IP-adres opvragen van de VM. Daarna wordt via SSH verbinding gemaakt en vervolgens de provisioningstappen uitgevoerd op het systeem.

A.3.4. Fase 4: Sprint 3 - configuratie

In de derde sprint wordt er ondersteuning toegevoegd voor een configuratiebestand in het TOML-formaat. De provisioner zal het configuratiebestand inlezen en verwerken, de gevraagde instellingen zullen het provisioninggedrag besturen.

A.3.5. Fase 5: Sprint 4 - functies

In de vierde sprint zullen nieuwe functies worden toegevoegd, zoals in het aanmaken en terugzetten van snapshots. Dit zorgt er voor dat de gebruiker snel terug zal kunnen gaan naar een stabiele toestand, zoals na een mislukte provisioningrun. Ook zal tijdens deze sprint meerdere boxes aangemaakt worden.

A.3.6. Fase 6: Sprint 5 - foutafhandeling

In de laatste sprint ligt de focus op het bouwen van foutafhandeling en rapportering. Fouten worden gelogd met duidelijke foutboodschappen en genoteerd in een log file. Als er een fout is moet er ook op een veilige manier een fallback-actie uitgevoerd worden, dit zorgt ervoor dat de gebruiker niet met een onbruikbare omgeving blijft zitten.

A.3.7. Planning

De totale duur van deze methodologie is gepland over een periode van 12 weken.

- Week 1-2: Rust aanleren
- Week 3-4: Sprint 1
- Week 5-6: Sprint 2
- Week 7-8: Sprint 3
- Week 9-10: Sprint 4
- Week 11-12: Sprint 5

A.4. Verwacht resultaat, conclusie

In dit onderzoek wordt er verwacht dat de ontwikkelde provisioner virtuele omgevingen van Windows en Linux kan opbouwen op een snellere en betrouwbaardere manier dan de huidige Vagrant provisioner. Dat de verkorte provisioning-tijd, minder mislukte runs en verlaagde manuele interventie er voor zorgt dat de virtuele omgeving betrouwbaarder opstart.

Voor de doelgroep, opleidingen die dat gebruik maken van provisioning, betekent dit dat lessen, oefeningen en projecten minder tijdverlies hebben door technische problemen. De bachelorproef zal niet alleen maar een proef-of-concept provisioner maken maar ook documentatie en een handleiding voor mogelijkheid van uitbreiding

Bibliografie

- Abdulhamid, S. M., Latiff, M. S. A., & Bashir, M. B. (2014). On-Demand Grid Provisioning Using Cloud Infrastructures and Related Virtualization Tools: A Survey and Taxonomy. <https://arxiv.org/abs/1402.0696>
- Ahmed, B. T. (2020). Virtualization Mechanisms and Tools: A Comprehensive Survey. *International Journal of Computer Science and Engineering*, 12(9), 1–15. <http://www.ijcse.net/docs/IJCSE20-09-04-022.pdf>
- Brikman, Y. (2025, 17 februari). *How to Manage Your Infrastructure as Code*. Geraadpleegd op 12 december 2025, van <https://www.gruntwork.io/books/fundamentals-of-devops/how-to-manage-your-infrastructure-as-code>
- Creeger, M. (2009). CTO Roundtable: Cloud Computing. *Communications of the ACM*, 52(8), 50–56.
- Hill, D. (2015). *A Comparison of Configuration Management Tools in Respect to Performance and Usability* [Master's thesis]. Dublin Institute of Technology. <https://davehill.ie/resources/Dave-Hill-Thesis.pdf>
- Ijaz, U. (2024). *A Comparative Lens on Infrastructure-as-Code Tools and Practices*. KTH. <https://www.diva-portal.org/smash/get/diva2:1941451/FULLTEXT01.pdf>
- Ismailzai, M. (2017, 8 december). *Infrastructure as Code: provisioning and configuration management with Vagrant, Terraform, and Ansible*. Geraadpleegd op 12 december 2025, van <https://www.mugo.ca/Blog/Infrastructure-as-Code-provisioning-and-configuration-management-with-Vagrant-Terraform-and-Ansible>
- Knuth, D. E. (1998). *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. Addison Wesley Longman Publishing Co., Inc.
- KPI Depot. (2024, 31 december). *Virtual Machine Provisioning Time*. Geraadpleegd op 12 december 2025, van <https://kpidepot.com/kpi/virtual-machine-provisioning-time>
- Lo, N.-W., Fan, P.-C., & Wu, T.-C. (2014). An efficient virtual machine provisioning mechanism for cloud data center. *2014 IEEE Workshop on Electronics, Computer and Applications*, 703–706. <https://doi.org/10.1109/IWECA.2014.6845718>
- Luo, C., Qiao, B., Chen, X., Zhao, P., Yao, R., Zhang, H., Wu, W., Zhou, A., & Lin, Q. (2020). Intelligent Virtual Machine Provisioning in Cloud Computing [Main track]. In C. Bessiere (Red.), *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20* (pp. 1495–1502). International Joint

- Conferences on Artificial Intelligence Organization. <https://doi.org/10.24963/ijcai.2020/208>
- Pollefliet, L. (2011). *Schrijven van verslag tot eindwerk: do's en don'ts*. Academia Press.
- Portworx. (2025). *Virtual Machine Provisioning at Enterprise Scale*. Portworx by Pure Storage. <https://portworx.com/wp-content/uploads/2025/10/wp-virtual-machine-provisioning-at-enterprise-scale.pdf>
- Robison, N. A., & Hacker, T. J. (2012). Comparison of VM deployment methods for HPC education. *Proceedings of the 1st Annual Conference on Research in Information Technology*, 43–48. <https://doi.org/10.1145/2380790.2380801>
- S P, U. K., & Antony D'Mello, D. (2018). Virtual Machine Provisioning and Resource Management Mechanisms for Dynamic Workloads. *2018 4th International Conference on Applied and Theoretical Computing and Communication Technology (iCATccT)*, 88–93. <https://doi.org/10.1109/iCATccT44854.2018.9001935>
- Steve Klabnik, C. K., Carol Nichols. (2025). *What is Ownership? - The Rust Programming Language*. Geraadpleegd op 28 januari 2026, van <https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>
- Wylde, M. (2023, 27 september). *Rust is the best language for data infra*. Geraadpleegd op 27 januari 2026, van <https://www.arroyo.dev/blog/rust-for-data-infra/>